

1. Modello Gerarchico (Web Browser): Il browser che utilizzi per navigare (come Chrome o Firefox) si basano su un sistema centralizzato chiamato PKI (Public Key Infrastructure).
 - Autorità Centrali: La fiducia è stabilita da enti terzi chiamati Certification Authorities (CA).
 - Fiducia "Dall'alto": Quando visiti un sito (es. della tua banca), il browser si fida del certificato del sito perché è stato firmato da una CA di cui il browser si fida già.
 - Ruolo dell'utente: L'utente è passivo; non deve verificare personalmente l'identità del proprietario del sito, poiché il software lo fa automaticamente tramite le CA pre-installate.
2. Modello Decentralizzato: Web-of-Trust (GPG): GPG utilizza invece il Web-of-Trust (Rete di Fiducia), dove la sicurezza non dipende da un'autorità centrale ma dalle relazioni dirette tra le persone.
 - Nessuna Autorità Centrale: Non c'è una "CA" suprema. La fiducia viene costruita "dal basso" attraverso le firme reciproche tra utenti.
 - Verifica Personale: Per fidarsi di una chiave, GPG suggerisce di incontrare la persona dal vivo, verificare un documento d'identità e controllare il fingerprint (impronta digitale) della chiave su carta.
 - Propagazione della Fiducia: Se io mi fido di una persona e firmo la sua chiave, e tu ti fidi di me, puoi scegliere di fidarti automaticamente anche delle chiavi che ho firmato io.

CyberSecurity

Edition: 2025-2026

Resources on [Virtuale](#)

T4 - Tutorial: Usage examples of GPG

Goal of this tutorial

The goal of this tutorial is to demonstrate a practical usage of the public-key encryption and to introduce the web-of-trust for establishing the authenticity of public-keys.

doesn't require to spend money (trust is built "from the ground up" through direct relationships between users. There is no central authority, Certificate Authority, that profits from emitting keys) and to identify yourself (The system gives you the freedom not to identify yourself (unlike banking systems or certified web domains), but the Web of Trust is designed so that security increases the more people voluntarily choose to identify themselves to one another.)

GPG introduction

Many software tools are based on public-key encryption. Among them, we will see [GnuPG](#) (GPG). GPG is a complete implementation of the OpenPGP ^{that is a} standard for sending and receiving secure emails. GPG implements both symmetric and asymmetric encryption ciphers and it is typically used for encrypting emails but it can also be used for encrypting generic input files (e.g., backups). There are many GUIs for facilitating the usage of GPG but in this specific case, we are interested in the baseline operations and procedures and therefore we use the command line.

Let's assume that there is a new user who is willing to set up a working GPG installation to communicate with friends. The first step is about generating a new key pair (i.e., public and private keys) for ^{his/her} their new identity.

All public-key algos are based on Complex Math Problems:

For example, New instance of Factorization problem: two big random prime numbers and multiply them (for generating random numbers, computer need entropy: difficult to generate because computer is deterministic). The factorization problem is the process of breaking down a large integer into the product of two prime numbers, a computationally very difficult task. Asymmetric Cryptography (RSA): It uses two keys: a public key and a private key. Security is based on the fact that, even if the public key (which contains the multiplied number) is known, it is impossible to derive the private key without knowing the prime factors p and q.

`gpg --gen-key` Creates 1 x key pair

The default parameters used by GPG are fair but they can be tuned in case enhanced security is required (e.g., longer key sizes). Depending on the OS used for running GPG (and the specific version of the OS kernel), this operation could require a significant amount of time. This is because, in some implementations, the kernel needs to collect "enough entropy" for providing secure random numbers that are required for the key generation. If you are interested in the details, more info is provided in the course slides.

After some time, the keys are generated and then tagged with a specific identifier. **Why does gpg ask the user to enter a passphrase? What is it used for?** ^(two: one for the public and one for the private)

→ The public key does not need to be protected by a passphrase because, by definition, it is intended to be distributed and known to everyone. The issue lies with the private key, which is stored locally in the user's home directory. If the private key were stored in plain text, anyone with access to the computer could use it to decrypt your messages. For this reason, GPG protects the private key using symmetric encryption, with a passphrase serving as the unlock key. This approach ensures a high level of confidentiality, but comes at the cost of usability: every time you need to perform an operation that requires the private key (such as decrypting a file), the software will require you to enter the passphrase to use the private key.

The traditional email protocol is not secure. Even today, only the communication channel between the user and the server, or/and between the servers themselves, is encrypted, while emails are stored in clear (therefore, no security properties are guaranteed for standard emails). If additional properties such as confidentiality, integrity, and authenticity are required, an extension of the protocol is necessary: this is precisely what OpenPGP and the GPG implementation provide.

It is a good idea to give an expiration date to the key pair generated: If a private key is stolen or leaked without your knowledge, an expiration date ensures that the attacker's access is time-limited. Without an expiry, a stolen key could be used indefinitely.

Why is it fundamental? The revocation certificate is essential for managing emergency scenarios where you lose control of your digital identity. The primary reasons why it is vital are: Compromise of the Private Key: If an attacker gains access to your private key, you can upload the revocation certificate to key servers to inform the world: "This key has been compromised, do not trust it anymore." Loss of the Passphrase or Key: Without the revocation certificate, your public key would remain on key servers forever, active but completely unusable, causing confusion for anyone trying to contact you. Changes in Personal Information: If you stop using the email address linked to that key, the certificate allows you to cleanly "retire" it. How does revocation work? Creation: You generate the certificate ahead of time while everything is still working perfectly by running the dedicated command. Secure Storage: You must store it separately from your computer. If you leave it on your computer and the hard drive breaks, you will lose both the key and the certificate needed to revoke it. Similarly, if the computer is compromised, an attacker could prevent you from deploying it. Deployment: If an emergency occurs, you take this file and import or send it to the key servers. From that moment on, anyone attempting to use your old public key will see a warning stating that the key has been revoked and is no longer secure.

It is possible to list all the public keys that are stored in our system (inside a "hidden" directory in the user's home directory) by using the following command:

```
gpg --list-public-keys
```

Identification of the public and private key of the pair is exactly the same:
- Digital Identity: This ID is used to "tag" your digital identity so that the software always knows which public key corresponds to which private key.
- Keyring Management: Whether you are viewing public keys (gpg --list-public-keys) or private keys (gpg --list-secret-keys), you will see the same ID associated with your identity (name/email).

A similar operation can be done for listing the available private keys:

```
gpg --list-secret-keys
```

Just after creating the new key-pair, it is important to immediately generate and secure the corresponding **revocation certificate**. Please, try to consider why a revocation certificate is fundamental. (In some recent versions of gpg the revocation certificate is automatically generated upon the key-pair generation). Please read carefully the warnings that are displayed when running the following command:

```
gpg --output revoke.asc --gen-revoke <KEY-ID>
```

If i have your revocation certificate, i cannot impersonate you (i needs your private key and the passphrase to unlock it), but i can create a denial-of-service to you

After sending the command, gpg asks if i am sure to do it for this key pair (yes or not) and the motivation for the creation of revocation certificate: No reason specified, Key has been compromised, Key is superseded (newly created keys), Key is no longer used, Cancel the revocation

the revocation certificate is signed by the private key of the key pair of the certificate to demonstrate that it is valid

Now our brand new public-key is ready to be used but... no one knows it! That's a problem! **We need a way for delivering our public-key to all our existing (and future) contacts.** A first simple approach would be to publish our public-key in a "secure way". **What do we exactly mean with "secure way" in this specific case? What are the relevant security properties to be considered?**

We need to carefully assess the threat model (for example, if I publish my public key on a website, I am trusting the administrators who manage the site: anyone who modifies the content of the webpage can replace my public key with that of an imposter)

We could make available the public-key in a webpage but the website must support the https protocol. Why? What's wrong with http? Minor problem: the generated public-key is a binary file. Likely a text file would be easier to be published on the web:

if you publish your key on a webpage, the site must use HTTPS rather than HTTP. HTTPS provides integrity and authenticity via the traditional PKI model (browser certificates), ensuring that users downloading the key from your site are actually connecting to your server and receiving the exact file you uploaded.

1) Access to your home directory is primarily restricted by the Operating System's File System Permissions. By default, GPG creates its storage directory with strict permissions, meaning only your specific user account and the system administrator (root) have read or write access to it. Is it secure? It is secure against other standard users on the same local machine. However, it is not entirely secure against a user with administrative/root privileges. Malware running on your system with your user permissions. Physical theft of the storage drive while the system is unencrypted.

```
From binary to text (public key)  
> gpg --export -a <KEY-ID> > mykey.asc
```

In the exported file, there is the text version of the public-key. The private-key remains stored (and well protected?) in my home directory. **Question: how is the access restricted to my home directory? Is it secure or not? Is the home directory content encrypted? How can I effectively manage the confidentiality of my private-key?**

Another approach for making my public-key available to my friends is to use a key server. That is an infrastructure that is specifically dedicated to the publication of public keys and their management.

Every time i create a key pair, i publish the public key to the server

```
> gpg --send-keys --keyserver <KEY-SERVER> <KEY-ID>
```

2) By Default: No. Standard operating system installations do not automatically encrypt the home directory unless you explicitly choose to enable a feature like Full Disk Encryption or home-directory-specific encryption. Even if the home directory itself is not encrypted, GPG compensates for this by automatically encrypting the private key file locally using symmetric encryption tied to your passphrase. Therefore, while the directory content is in plain text, the private key itself is not.

When you share your public key, the primary threat is a Man-in-the-Middle (MitM) attack, where an attacker replaces your public key with their own. If your contacts accidentally use the attacker's key, the attacker will be able to decrypt messages intended for you. To prevent this, two relevant security properties must be considered: 1. Integrity: What it means: Ensuring that the public key is not altered, tampered with, or replaced by a malicious actor during transit or while hosted on a server. Why it matters: If an attacker modifies even a single character of your key (or swaps it entirely), the key becomes useless or, worse, compromised. 2. Authenticity (Identity Binding): What it means: Providing a reliable way for your contacts to verify that the public key genuinely belongs to you and not to an imposter. Why it matters: Anyone can generate a key pair using your name and email address. Authenticity ensures your contacts can prove the cryptographic bond between your real-world identity and that specific <KEY-ID>.

3) To maximize the confidentiality of your private key, you should implement security layers beyond GPG's default behavior:
- Use Disk Encryption: Ensure your entire operating system or home directory is encrypted at rest. This protects your private key file from being extracted if your laptop or hard drive is physically stolen.
- Choose a Strong Passphrase: Since GPG uses symmetric encryption to lock your private key, the strength of that protection relies entirely on your passphrase. Use a long, complex passphrase that cannot be easily brute-forced.
- Isolate Your Key (Hardware): For advanced security, you can move your private key out of your home directory entirely and store it on a dedicated cryptographic hardware device.
- Keep System Permissions Intact: Periodically verify that the permissions of your GPG directory have not been accidentally altered or made readable by global users.

No Identity Verification: The key server is purely an automated database. It does not check if you are a real human being, nor does it check if the email address or name attached to the key actually belongs to you. Anyone Can Impersonate Anyone: If a malicious actor wants to generate a fake key pair using your exact name and your exact email address, the key server will happily accept it. It does not send a verification email or ask for an ID. The Only Technical Check: The server only verifies that the uploaded public key is structurally valid and mathematically matches its own self-signature.

Why? Otherwise, nobody will be able to create an identity and to remain anonymous (it is a choice: you can implement verifications but it is a certificate authority at this point)

This operation uploads the newly created public-key to a specified key server. We will **NOT execute this operation** since the key-pair that we have created is not related to a real human being but it is only for testing purposes.

Obviously, it is also possible to query the key server for finding already existing public keys. For example, the query could be based on the email address that is linked to the public-key.

> `gpg --search-keys 'g.dangelo@unibo.it'` You can specify the key server

If the output of the command reports the public-key that I'm looking for then I can import it into my keyring. **Warning: think carefully about the security implications of this action.** In the following, we will discuss this issue again.

The Risk of Identity Theft (Impersonation): As established, public key servers do not verify the ownership of an identity. Anyone can generate a key pair using your email address (no validation of email address). If you import a malicious key thinking it belongs to your professor, you have no way of knowing it is fake just by looking at the email address attached to it.

After importing the public-key, I can verify that it is now included in my keyring.

> `gpg --list-public-keys`

To safely counter this implication, GPG separates importing a key from trusting a key:
- Check the Fingerprint: After importing, you must never use the key until you verify its unique cryptographic fingerprint through an out-of-band, secure channel (e.g., seeing it printed on a syllabus, or checking it directly with the owner or phone call).
- Sign the Key: Once you are 100% sure the key is authentic, you manually sign it with your own private key. This explicitly tells GPG: "I have verified this identity; it is safe to use."

Now, we use the newly imported key to encrypt a document that needs to be delivered in a confidential way (to the owner of the public-key):

> `gpg --encrypt --recipient 'g.dangelo@unibo.it' foo.txt`

After the text file has been encrypted using the recipient's public key (foo.txt.gpg: binary file), I can send the message via email. If you add the --armor option (or its abbreviation -a): GPG will generate a file with the .asc extension (e.g., foo.txt.asc) instead of .gpg. This file is a readable plain text file (ASCII).

Clearly, it is possible to also encrypt binary data and not only text files.

If it is not necessary to hide the content of the message but only to authenticate its originator (and to guarantee its integrity) then we can proceed in this way: No confidentiality of the content of the message

> `gpg --output foo.txt.sig --sign foo.txt`

Using the private key of the sender for creating a new message with a sign at the bottom of the message (if it is available the public key of the sender, the recipient can validate authenticity of the message by the sender)

As seen in the slides about public-key encryption, in this case, the sender does not need to use the public-key of the recipient (but the recipient will need the public-key of the sender to validate the authenticity of the received message).

It is also possible to combine both operations to get confidentiality, authenticity, and integrity:

> `gpg --encrypt --sign --recipient 'g.dangelo@unibo.it' foo.txt`

Encrypted message with all the three properties

Web-of-Trust

At its core, the absolute biggest problem in public-key cryptography is not the encryption itself, it is identity verification. Anyone can generate a public key with the name "Elon Musk" and upload it to a server. The Web-of-Trust is a decentralized framework designed specifically to prove that "this specific public key mathematically belongs to this specific real-world person" without relying on a centralized authority (like a company or a government).

Now we can consider again the problem related to the authenticity of the public-key that we used for encrypting our message to preserve confidentiality. The mechanism that is typically used by gpg is **different from the approach followed in web browsers** (that we will see in the

When you create a public key and publish it on a public key server, the server acts merely as a directory; it does not function as a Certification Authority (CA). This means the key server completely skips verifying the binding between a person's real-world identity and their public key. Consequently, the only way to genuinely validate a recipient's public key and confirm it belongs to the correct person is to verify this identity binding in person (In Real Life). While checking a physical ID and a printed cryptographic fingerprint is highly reliable when face-to-face, establishing this level of authenticity online is incredibly difficult due to the risk of interception and impersonation. Once you have successfully verified someone in person, you can digitally sign their public key and upload your signature back to the network, effectively spreading the information to others that this identity has been trusted and verified by you.

following of this course) and it is called “**web-of-trust**”. The basic idea of the web-of-trust is that after carefully checking the identity of the people that we know (in real life) and their matching with their corresponding public-key, then we can validate this match by signing their public-key (using our private-key). In practice, signing a key of someone has the effect of extending our web-of-trust and moreover, maybe we can accept to trust someone that is unknown to us but that has been validated by one or more friends of ours. Please note that, in this case, the number of friends that validate the public-key of someone that I don’t know affects my perception of trust. Based on what I know about the person associated with that key, I have different degrees of trust (I have more trust on my friends than on friends of friends)

Since it proves quite inconvenient to bring with us the full printed version of our public-key (i.e., it is quite long), it is preferred to use the fingerprint (i.e., a hash value) of the public-key. This is generated using the following command:

→ `gpg --fingerprint <KEY-ID>`

The suggested procedure to follow is: print out on paper the fingerprint and the identifier of the public-key. All of this is then provided to the person who is willing to sign our public-key. This operation should happen only after the verification of an identification document.

When all these operations are completed then it is possible to get the key to be signed using:

→ `gpg --keyserver <keyserver> --recv-keys <KEY-ID>`

Now, after a final check to verify that the information on the public-key is as expected, it is possible to proceed with the key signature:

→ `gpg --sign-key <KEY-ID>`

At this point, the signing operation is stored only locally and it needs to be propagated to a key server for really building a public web-of-trust:

→ `gpg --keyserver <keyserver> --send-key <KEY-ID>`

It is possible to list all the signatures that have been collected by a specific public-key using the following command:

→ `gpg --list-sigs <KEY-ID>` In the traditional implementation, it was possible. Now, in many server, is no more possible

Note: if this command reports only self-signatures for a given key then this is due to a change of behavior in the signing of keys that has been implemented to mitigate the problem of fake keys submitted to key servers. The actual procedure being implemented to create a web-of-trust is

more complex than the one reported in this tutorial but the theoretical framework discussed here is still valid.

Using GPG for symmetric encryption

Even if GPG is typically used for public-key encryption, it also implements some symmetric ciphers. For example:

→ `gpg --output foo.txt.gpg --symmetric foo.txt`

This operation encrypts an input file using the default symmetric cipher provided by GPG. Clearly, in this case, GPG asks for a secret key that needs to be shared with the recipient of the encrypted file.

The different symmetric ciphers provided by GPG can be listed with:

→ `gpg --version`

AGE Modern alternative of GPG: encrypt file using symmetric/public-key encryption (has also post-quantum algo)

The usability of GPG is quite limited^(terrible) and therefore many other tools have been proposed for encrypting messages and files. In many cases, they also have the goal to reduce the attack surface that in the case of GPG is too large. Among them, [age](#) is a very promising tool that is implemented using the Go programming language. It is cross-platform (Windows, GNU/Linux, Mac) and it is already part of many Linux distributions. For a list of usage examples please see the [age](#) web page.

References for more detail about GPG and Web-of-trust

- [The GNU Privacy Handbook](#)
- [Web of trust \(Wikipedia\)](#)